

Design Rationale Assignment 2 (Req 1)

Name: Pemudi Hiruni Halgahawatta Liyanaarachchi
Monash ID: 34246517
Unit: FIT 2099 Object Oriented Design and Implementation

Requirement 1: The Life in the Valley - OmenSheep Reproduction

The goal of this requirement was to introduce a new feature to SpiritGoat and OmenSheep; an ability to produce offspring.

OmenSheep is required to produce an Egg ('o') every 7 turns to the game map. When this egg is left on the ground for 3 turns, an offspring, a new OmenSheep, will hatch from the egg. The farmer is able to pick up the egg and add it to their inventory; once added egg will not hatch and the farmer is able to consume the egg to which his health points will increase by 10 points.

While the production of SpiritGoat is handled and implemented by a teammate, this design rationale section will handle the OmenSheep.

Approach: Design and Implementation

Classes/Interfaces/Enums created/updated:

ProduceBehaviour, OmenSheepEgg, OmenSheep (updated)

As the OmenSheep should have the ability/behaviour to produce an egg, a ProduceBehaviour class, which implements Behaviour, was created. The OmenSheep is passed in as a producer into the ProduceBehaviour() method as OmenSheep was made to implement Producible.

The ProduceBehaviour class follows the **Single Responsibility Principle (SRP)** as it decides whether an actor can perform a production action. Actors such as SpiritGoat and OmenSheep can use ProduceBehaviour without needing to modify it, which showcases the **Open/Closed Principle (OCP)**.

Also ProduceBehaviour depends on the abstraction (Producible) and not on a concrete class like OmenSheep, it has to be a Producible element. This follows the **Dependency Inversion Principle (DIP)** which promotes decoupling and reuse of code.

OmenSheep class was also updated to track turn count and spawn an egg through the canProduce() method.

A subclass of Egg was created called OmenSheepEgg, which manages ground aging, hatching of the egg and providing an increase in health points once consumed. This class

follows the **Liskov Substitution Principle (LSP)** where with extended functionality as an egg, it is able to hatch into OmenSheep and is edible by the Player.

Once implemented, for the egg to be picked and consumed by the Player, the `getAllowableActions()` and `eat()` actions were overridden and added a feature where Farmers health points are increased by 10 points.

Functionality of approach:

In OmenSheep, a countdown (`eggTimer`) is incremented every turn. Once it hits 7, the `produce()` method spawns an `OmenSheepEgg` at the current location and resets the timer. Through the `shouldHatch()`, the egg tracks the remaining turns on the ground via `groundAge` and hatches it after 3 ticks.

Advantages and disadvantages of design approach:

Advantages:

The design for Egg is flexible as the eggs can evolve with different hatching or consumption features.

Disadvantages:

The `groundAge` and hatching logic in egg are still time-based only. Also, the egg does not track the producer/owner, which may limit narrative control for the game.

Alternative Design Approach:

An alternative approach would be to move all production and hatching logic into a central class, however this may centralize too much responsibility in one class and violate the SRP principle. Therefore, the approach provided adheres to SOLID principles and provides a cleaner solution.